Full-stack assignment · Django REST Framework + React

"FlashDeck" — Your Own Spaced-Repetition Study App

Build the tool you will use to revise for your own interviews: decks of cards that come back exactly when you are about to forget them.

Type: Full-stack — Django REST Framework backend + React frontend
Repositories: 2 separate public GitHub repos (one backend, one frontend)
Duration: 4–5 days · **Submission:** see section 11
Scope: 2 models · 8 endpoints · 7 screens

0. Read this first

You are building **FlashDeck** — a private flashcard app that decides *when* to show you each card. You write a question on the front and the answer on the back. Every card sits in one of five boxes. Answer it correctly and it climbs a box and disappears for longer; miss it and it drops straight back to Box 1 and returns the same day. Each user logs in and sees only their own decks.

Two reasons this project and not another one: it is small enough to finish in 4–5 days, and you will genuinely use it — one deck of Django questions, one of React questions — while you prepare for interviews. That makes it a much better CV entry than another to-do app.

Two kinds of requirements:

- **REQUIRED** (B-1 … B-6, F-1 … F-7, S-1 … S-3) — this is all you are graded on. It is deliberately small: two models, eight endpoints, seven screens. Finish these and you get full marks.
- **OPTIONAL** (O-1 … O-16) — worth **zero** marks. Skipping all of them costs you **nothing**. But the required list alone is a class exercise; the optional list is what turns it into a project a recruiter opens twice. Push the required part first, then add whichever optional items you like over the following days.

Do not start the optional list before the required list is finished and pushed.

The frontend design is yours. This brief tells you what information must be on each screen and gives you a suggested layout — it does not tell you how it should look. Section 6 explains exactly how much freedom you have (short version: a lot).

You may change the domain. Vocabulary drills for IELTS, kanji, medical terms, driving-test questions, Quran memorisation tracking — all fine. The rule: **one parent model owned by the user, one child model, one 5-step field that only the server may change**, and the same required list. Write your chosen domain at the top of both READMEs.

1. Tech stack

Backend — repo #1

- Python 3.10+, Django 4.2+ / 5.x
- Django REST Framework
- `djangorestframework-simplejwt` (JWT auth)
- `django-cors-headers`
- `django-filter`
- SQLite is fine
- `python-decouple` or `django-environ` for the secrets

Frontend — repo #2

- React 18 + Vite
- `react-router-dom` v6

- `axios` — one configured instance, not bare `fetch` calls scattered around components
- Styling: **your choice** — Tailwind, plain CSS, CSS Modules, styled-components. Pick one and use it consistently.
- Optional and allowed: React Query, react-hook-form, react-hot-toast, lucide-react / react-icons, recharts

Not allowed

- A downloaded dashboard template or theme, re-skinned and submitted as your own work
- A component library used as the whole design (a few primitives are fine; a ready-made admin panel is not)
- Hard-coded data in React. Every value on the screen comes from your API.
- Next.js or any framework other than Vite + React
- Firebase / Supabase in place of your own Django API
- **An SRS or flashcard package that schedules the cards for you.** The box logic in B-5 is the heart of this project — it must be your own Python, written by you.

2. Repositories

Two public repos:

```
<your-name>-flashdeck-backend    ->  Django + DRF
<your-name>-flashdeck-frontend   ->  React + Vite
```

- `.gitignore` must exclude `venv/`, `__pycache__/`, `db.sqlite3`, `.env`, `media/`, `node_modules/`, `dist/`
- Backend: at least 6 meaningful commits. Frontend: at least 6. Not one giant "initial commit".
- Each README links to the other repo.

A plan that fits the deadline

Day	Backend	Frontend
1	B-1 models + admin, B-2 register / login working in Postman	Vite project, routing, the axios instance (F-1)
2	B-3 decks viewset + owner scoping	F-2 login / register / ProtectedRoute wired to the API
3	B-4 cards viewset, filter, search, pagination	F-4 deck list + deck detail with the three states
4	B-5 study + review (the box logic), B-6 stats	F-6 study screen, F-5 forms
5	README, deploy or polish (optional)	F-3 dashboard, F-7 delete, screenshots, README

Build the backend endpoint first, then the screen that uses it. Do not build all the screens against fake data and try to connect them at the end — that is where projects die.

PART A — REQUIRED (this is what is graded)

3. Backend

B-1 The models

Two models in a parent → child relation. `Deck` belongs to a user; `Card` belongs to a deck.

`Deck`

Field	Type	Notes
owner	FK -> User	on_delete=CASCADE, related_name="decks"
title	CharField(120)	required, unique per owner (case-insensitive)
description	TextField	blank

Field	Type	Notes
subject	CharField + choices	PROGRAMMING / LANGUAGE / ACADEMIC / INTERVIEW / OTHER, default OTHER
is_archived	BooleanField	default False
created_at, updated_at	auto	auto_now_add / auto_now

`Card`

Field	Type	Notes
deck	FK -> Deck	on_delete=CASCADE, related_name="cards"
front	TextField	required — the question
back	TextField	required — the answer
hint	CharField(200)	blank
box	PositiveSmallIntegerField	1 … 5, default 1 — **server-owned**
next_review_at	DateTimeField	default=timezone.now
last_reviewed_at	DateTimeField	null=True, blank=True
times_reviewed	PositiveIntegerField	default 0
times_correct	PositiveIntegerField	default 0
created_at, updated_at	auto	auto_now_add / auto_now

- `Deck.Meta`: `ordering = ["-created_at"]`. `Card.Meta`: `ordering = ["box", "next_review_at"]`.
- Use `models.TextChoices` for `subject`.
- `Deck.__str__` returns the title; `Card.__str__` returns the first 40 characters of `front`.
- Register both models in the Django admin. In `CardAdmin` set `list_display` to show the deck, the box and the due date.

> **Two rules that fail B-1 on their own**
>
> `owner` **is never sent by the client.** It is set in `perform_create()` from `request.user`, and it is in `read_only_fields`.
>
> `box`, `next_review_at`, `last_reviewed_at`, `times_reviewed` **and** `times_correct` **are never sent by the client either.** They only ever change inside the review action (B-5). If `PATCH /api/cards/7/` with `{"box": 5}` moves a card to Box 5, B-1 is failed.

One card object looks like this — your frontend is built against exactly this shape:

```
{
  "id": 41,
  "deck": 3,
  "deck_title": "Django ORM",
  "front": "What does select_related() do?",
  "back": "Follows forward FK relations with a SQL JOIN, in one query.",
  "hint": "N+1",
  "box": 3,
  "next_review_at": "2026-09-08T06:00:00Z",
  "last_reviewed_at": "2026-09-05T06:12:44Z",
  "times_reviewed": 6,
  "times_correct": 5,
  "is_due": false,
  "created_at": "2026-08-21T09:14:02Z",
  "updated_at": "2026-09-05T06:12:44Z"
}
```

And a deck, with the two counts the list screen needs:

```
{
  "id": 3,
  "title": "Django ORM",
  "description": "Everything I keep forgetting about querysets.",
  "subject": "PROGRAMMING",
  "is_archived": false,
  "card_count": 24,
  "due_count": 7,
  "created_at": "2026-08-21T09:14:02Z",
  "updated_at": "2026-09-01T11:02:44Z"
}
```

`deck_title`, `is_due`, `card_count` and `due_count` are computed on the server and are read-only. Never make the browser count them.

B-2 Auth

Method	Endpoint	Who	Body	Returns
POST	/api/register/	anyone	username, email, password	201 + the created user (no password)
POST	/api/login/	anyone	username, password	200 + { "access": "...", "refresh": "..." }
POST	/api/token/refresh/	anyone	refresh	200 + a new access

- The password field is `write_only` and hashed — create the user with `User.objects.create_user()` or `set_password()`. A register response that echoes the password fails B-2.
- Minimum password length 6, `username` unique, `email` required.
- A bad register returns **400** with the DRF error shape your frontend has to read: `{"username": ["A user with that username already exists."]}`
- A bad login returns **401** with `{"detail": "No active account found with the given credentials"}`.
- Set a sane access-token lifetime (e.g. 1 day while developing) in `SIMPLE_JWT`.

B-3 Decks CRUD — owner-scoped

One `ModelViewSet` registered on a router.

Method	Endpoint	Success	Notes
GET	/api/decks/	200	paginated, own decks only
POST	/api/decks/	201	owner set from the token
GET	/api/decks/<id>/	200	own deck only
PUT / PATCH	/api/decks/<id>/	200	own deck only
DELETE	/api/decks/<id>/	204	its cards go with it (CASCADE)

- `permission_classes = [IsAuthenticated]` — a request with no token returns **401**.
- `get_queryset()` returns **only** `Deck.objects.filter(owner=self.request.user)`.
- Because of that, another user's id returns **404** — not their data, and not 403. Create two accounts and test this before you submit; this single check is worth more marks than any feature in the optional list.
- Search by title with `?search=`, filter with `?subject=` and `?is_archived=`.
- Two decks with the same title for the same user return **400** with the message under `title` — see hint 7.3. A 500 from a database `IntegrityError` is not acceptable.

```
class DeckViewSet(viewsets.ModelViewSet):
    serializer_class = DeckSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        now = timezone.now()
```

```
        return (Deck.objects
                .filter(owner=self.request.user)
                .annotate(
                    card_count=Count("cards", distinct=True),
                    due_count=Count("cards", filter=Q(cards__next_review_at__lte=now),
distinct=True),
                ))

    def perform_create(self, serializer):
        serializer.save(owner=self.request.user)
```

Do the two counts with `annotate()`, in one query. Looping over the decks in Python and calling `.count()` per deck is the N+1 problem, and it is visible in the Django debug output.

B-4 Cards CRUD — scoped through the deck

Method	Endpoint	Success	Notes
GET	/api/cards/?deck=3	200	paginated, 10 per page
POST	/api/cards/	201	body carries `deck`, `front`, `back`, `hint`
GET	/api/cards/<id>/	200	own card only
PUT / PATCH	/api/cards/<id>/	200	text fields only
DELETE	/api/cards/<id>/	204	own card only

A card has no `owner` column. Ownership is reached **through the deck**:

```
def get_queryset(self):
    return Card.objects.filter(deck__owner=self.request.user).select_related("deck")
```

> **The check that is marked first**
>
> `deck` *is* sent by the client — so it must be validated. Without `validate_deck()`, user B can post a card into user A's deck simply by sending A's deck id. Reject it with a 400, under the `deck` key. The code is in hint 7.3.

Filter, search, ordering

Query param	Example	Behaviour
deck	?deck=3	exact match — the deck detail screen always sends it
box	?box=1	exact match, 1 … 5
search	?search=queryset	case-insensitive match on `front` or `back`
ordering	?ordering=-created_at	`created_at`, `box`, `next_review_at`, ascending or descending

```
    filter_backends = [DjangoFilterBackend, filters.SearchFilter, filters.OrderingFilter]
    filterset_fields = ["deck", "box"]
    search_fields = ["front", "back"]
    ordering_fields = ["created_at", "box", "next_review_at"]
    ordering = ["box", "next_review_at"]
```

They must combine: `/api/cards/?deck=3&box=1&search=orm&ordering=-created_at&page=2` returns one sensible result set, still scoped to the logged-in user.

Pagination

10 per page (`?page=2`). The list response must be the standard DRF envelope, because the frontend needs `count` to draw the page numbers:

```
{
  "count": 24,
```

```
  "next": "http://127.0.0.1:8000/api/cards/?deck=3&page=2",
  "previous": null,
  "results": [ ... ]
}
```

An out-of-range page returns 404 — handle it in the frontend by resetting to page 1.

B-5 Study and review — the box ladder

This is the part that makes the project worth building. Everything else is CRUD.

```
  correct ->        correct ->        correct ->        correct ->
+---------+       +---------+       +---------+       +---------+       +---------+
|  Box 1  |  ---> |  Box 2  |  ---> |  Box 3  |  ---> |  Box 4  |  ---> |  Box 5  |
+---------+       +---------+       +---------+       +---------+       +---------+
 again today       +1 day            +3 days           +7 days           +16 days
     ^                                                                       |
     +------------------------ missed: back to Box 1 -------------------------+
```

A missed card drops straight back to Box 1 from wherever it was, and is due again in the same session. Box 5 is as high as a card goes.

The two endpoints

Method	Endpoint	Body	Returns
GET	/api/decks/<id>/study/	—	up to 20 cards from that deck that are due now
POST	/api/cards/<id>/review/	{"correct": true}	200 + the updated card

`GET /api/decks/3/study/` is **not paginated** — it returns one object the study screen can consume directly:

```
{
  "deck": { "id": 3, "title": "Django ORM" },
  "due_count": 7,
  "cards": [ /* up to 20 full card objects, oldest due first */ ]
}
```

- Due means `next_review_at <= timezone.now()`, and the deck must belong to the caller — someone else's deck id returns 404.
- Order by `next_review_at` ascending, cap at 20 with a slice (`[:20]`).
- Nothing due returns `"due_count": 0` and an empty `cards` list with a **200**, not a 404. An empty study session is a normal day, not an error.

`POST /api/cards/41/review/` applies the ladder:

Answer	New box	next_review_at
correct: true	min(box + 1, 5)	now + INTERVALS[new box]
correct: false	1	now (due again today)

- `INTERVALS = {1: 0, 2: 1, 3: 3, 4: 7, 5: 16}` — days, as a module-level constant, not five `if` branches.
- Both answers set `last_reviewed_at = timezone.now()` and increment `times_reviewed`. Only a correct answer increments `times_correct`.
- A missing or non-boolean `correct` returns **400**: `{"correct": ["This field is required and must be true or false."]}`. Note that `"correct": "false"` is a *string*, and a string is truthy in Python — validate the type, do not just read the key.
- Someone else's card id returns **404**, because the action uses the same scoped `get_queryset()`.
- Use `timezone.now()`, never `datetime.now()`. Full code is in hint 7.4.

B-6 Stats endpoint

`GET /api/stats/` returns the numbers **for the logged-in user only**:

```
{
  "decks": 4,
  "cards": 128,
  "due_now": 17,
  "mastered": 23,
  "reviewed_today": 40,
  "accuracy": 78,
  "boxes": { "1": 40, "2": 30, "3": 20, "4": 15, "5": 23 }
}
```

- `mastered` is the number of cards in Box 5; `reviewed_today` counts cards whose `last_reviewed_at` falls on today's date.
- `accuracy` is an **integer percentage**: `round(100 * sum(times_correct) / sum(times_reviewed))`. When nothing has been reviewed yet this is **0** — guard the division. A `ZeroDivisionError` on a brand-new account is a 500 on the first screen a grader opens.
- Build `boxes` with one `values("box").annotate(Count("id"))` query, not five separate `.count()` calls, and fill in the missing boxes with 0.
- **Every key must always be present**, including all five box keys, even when the count is 0 — otherwise the dashboard renders `undefined`.
- Requires a token; anonymous gets 401.

4. Frontend

F-1 Project shape and the axios instance

A suggested folder layout (rename freely, but keep the API calls out of the components):

```
src/
  api/
    client.js        # THE single axios instance
    decks.js         # one function per endpoint
    cards.js
    auth.js
  auth/
    AuthContext.jsx  # or a simple useAuth hook
  components/
    ProtectedRoute.jsx  Navbar.jsx  StatCard.jsx
    DeckCard.jsx  CardRow.jsx  BoxBadge.jsx  ConfirmModal.jsx
    Loader.jsx  EmptyState.jsx  ErrorState.jsx
  pages/
    Login.jsx  Register.jsx  Dashboard.jsx
    DeckList.jsx  DeckDetail.jsx  DeckForm.jsx
    CardForm.jsx  Study.jsx
  App.jsx  main.jsx
```

The axios instance — this is the part most people get wrong, so here it is in full.

```
// src/api/client.js
import axios from "axios";

const client = axios.create({
  baseURL: import.meta.env.VITE_API_URL,   // http://127.0.0.1:8000/api
  headers: { "Content-Type": "application/json" },
});

// 1. attach the token to every request
client.interceptors.request.use((config) => {
  const token = localStorage.getItem("access");
  if (token) config.headers.Authorization = `Bearer ${token}`;
```

```
  return config;
});

// 2. one place to handle an expired / missing token
client.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("access");
      localStorage.removeItem("refresh");
      if (window.location.pathname !== "/login") window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);

export default client;
```

`.env` in the frontend repo — and `.env.example` committed:

```
VITE_API_URL=http://127.0.0.1:8000/api
```

Vite only exposes variables that start with `VITE_`, and it reads `.env` **once at startup** — restart `npm run dev` after you change it.

Wrap the endpoints in a tiny API layer so no component ever writes a URL string:

```
// src/api/cards.js
import client from "./client";

export const listCards = (params) =>
  client.get("/cards/", { params }).then((r) => r.data);

export const createCard = (payload) =>
  client.post("/cards/", payload).then((r) => r.data);

export const updateCard = (id, payload) =>
  client.patch(`/cards/${id}/`, payload).then((r) => r.data);

export const deleteCard = (id) => client.delete(`/cards/${id}/`);

export const reviewCard = (id, correct) =>
  client.post(`/cards/${id}/review/`, { correct }).then((r) => r.data);

// src/api/decks.js
export const listDecks = (params) => client.get("/decks/", { params }).then((r) => r.data);
export const getStudySession = (id) => client.get(`/decks/${id}/study/`).then((r) => r.data);
export const getStats = () => client.get("/stats/").then((r) => r.data);
```

Three details that will cost you an evening if you miss them

- **Trailing slashes.** Django's `APPEND_SLASH` redirects `/api/cards` to `/api/cards/`, and a redirected POST loses its body. Always write the trailing slash.
- `baseURL` **ends without a slash, every path starts with one** (`http://127.0.0.1:8000/api` + `/cards/`). Doubling or dropping the slash gives you a 404 that looks like a backend bug.
- `params` **is an object, not a string.** `client.get("/cards/", { params: { deck, search, box, page } })` — axios drops `undefined` values for you, so an empty filter simply disappears from the URL.

F-2 Auth flow and routes

- **Login** posts to `/login/`, stores `access` and `refresh`, and navigates to `/`.
- **Register** posts to `/register/`, and on success either logs the user in or sends them to `/login` with a success message. Field errors from the 400 response are shown under the fields.

- `ProtectedRoute` wraps every private route and redirects to `/login` when there is no token. Typing `/decks` while logged out must not flash the page before redirecting.
- **Navbar** shows the logged-in username and a **Logout** button that clears both tokens and returns to `/login`.
- Refreshing the browser on a protected page must keep you logged in — read the token from `localStorage` on startup, do not keep it only in React state.

Required routes:

Route	Access	Screen
/login	public	login form
/register	public	register form
/	protected	dashboard
/decks	protected	deck list + search + filter + pagination
/decks/new	protected	deck form
/decks/:id	protected	deck detail — the cards in that deck
/decks/:id/edit	protected	deck form (the same component)
/decks/:id/cards/new	protected	card form
/cards/:id/edit	protected	card form (the same component)
/decks/:id/study	protected	study screen

F-3 Dashboard

- Four stat cards from `GET /api/stats/`: **Due now**, Total cards, Mastered (Box 5), Accuracy %. The numbers come from the API — never from counting an array in the browser.
- A **box distribution** row: five small bars or chips built from `stats.boxes`, so you can see at a glance how much of the pile is still in Box 1.
- The decks with cards due, each showing its `due_count` and a **Study** button that goes straight to `/decks/:id/study`.
- A primary **New deck** button.
- Loading and error states here too. A new user with zero decks must see a friendly empty state, not four cards full of `undefined`.

F-4 Deck list and deck detail

/decks — the list

- A search box bound to `?search=` and a subject dropdown bound to `?subject=` (including an "All subjects" option).
- Each deck shows title, subject, `card_count`, `due_count`, and Study / Edit / Delete actions.
- Pagination built from `count` and the page size.

/decks/:id — the cards in one deck

- Fetches `/api/cards/?deck=:id`. The deck title and description are shown at the top.
- A search box bound to `?search=` and a box filter bound to `?box=` (All / 1 / 2 / 3 / 4 / 5).
- Each row shows the front, a truncated back, a **box badge**, when it is next due ("due now" / "in 3 days"), and Edit + Delete.
- Pagination from `count`. **Changing the search text or the box filter resets the page to 1** — forgetting this shows an empty page 3 and looks like a bug.

The three states, all three required, on both screens:

- **loading** — a spinner or skeleton rows, not a blank screen
- **empty** — "No cards in this deck yet" plus a button that opens the add form (and a different message when

a filter returns nothing: "No cards match this filter")
- **error** — "Could not load this deck." plus a **Retry** button. Test it by stopping the backend.

F-5 Add / edit forms

Two form components, each used by both its create and its edit route.

- **Deck form** — title (required), description, subject dropdown.
- **Card form** — front (required), back (required), hint. The deck is taken from the route (`/decks/:id/cards/new`), not typed by the user. Do not put `box` or `next_review_at` on the form; they are the server's.
- In edit mode, load the record first and pre-fill every field. Show a loader while it is fetching.
- On submit, disable the button and show "Saving…" so nobody double-submits.
- When the API answers **400**, show each message **under the field it belongs to**. DRF returns `{"title": ["You already have a deck with this title."]}` — an object of arrays. One red "Something went wrong" toast is not enough.
- On success: navigate back and show a success message. After adding a card, staying on the form with the fields cleared ("Save and add another") is a nice touch — 20 cards is a lot of round trips otherwise.

```
const [errors, setErrors] = useState({});

async function handleSubmit(event) {
  event.preventDefault();
  setSaving(true);
  try {
    id ? await updateCard(id, form) : await createCard({ ...form, deck: deckId });
    navigate(`/decks/${deckId}`);
  } catch (err) {
    if (err.response?.status === 400) setErrors(err.response.data);
    else setErrors({ detail: "Something went wrong. Please try again." });
  } finally {
    setSaving(false);
  }
}

// under the input
{errors.front && <small className="error">{errors.front[0]}</small>}
```

F-6 The study screen — the one that sells the project

`/decks/:id/study`. On mount it calls `GET /api/decks/:id/study/` once and keeps the returned cards in state. It does **not** refetch after every answer.

- Show the **front only**, plus the hint if there is one, and a **Show answer** button. Nothing on the screen may reveal the back before the user asks for it — no `opacity: 0` text they can select, no answer in a `title` attribute.
- After the flip, two buttons: **Got it** and **Missed**. Each posts to `/api/cards/:id/review/` and then moves to the next card.
- Progress is visible: "4 / 12", a thin progress bar, and the current card's box.
- Disable both buttons while the POST is in flight, so a double click cannot review the same card twice.
- **The finish state:** "Session done — 12 reviewed, 9 correct, 75%", with *Back to deck* and *Study again*. *Study again* refetches, because the cards you missed are due again immediately.
- **The empty state:** "Nothing due in this deck right now." plus a link back. This screen is empty most days *by design* — an app that shows a blank white page when there is nothing due looks broken.
- Loading and error states as everywhere else.

> **Why one fetch, not one per answer**
>
> A study session is a queue you were handed at the start. Refetching after every answer re-sorts the queue

> under the user's feet and shows the card they just missed again immediately. Fetch once, walk the array with an index, and only refetch when they press *Study again*.

F-7 Delete

- Deleting a deck and deleting a card both go through **your own confirmation modal**. `window.confirm()` is not accepted.
- The modal names what is being deleted, has Cancel and Delete, and closes on Cancel or Escape.
- Deleting a deck must say what it takes with it: **"Delete 'Django ORM' and all 24 cards in it? This cannot be undone."**
- After a successful delete the list updates **without a full page reload** — refetch the current page, or remove the row from state.
- If the deleted row was the last one on the page, go back one page instead of showing an empty list.

5. Submission requirements

S-1 No secrets in git. `.env` is ignored in both repos; each repo ships a `.env.example`. A committed `SECRET_KEY`, `db.sqlite3` or `node_modules/` is a deduction.

S-2 CORS configured so the Vite dev server can call the API, and `VITE_API_URL` read from the frontend `.env` — no `http://127.0.0.1:8000` hard-coded in a component.

S-3 Both READMEs written (see section 11).

6. UI structure — a starting point, not a rule

Below is one layout that works. **Change anything you want**: use a table instead of cards, move the filters into a sidebar, merge the dashboard into the deck list, add a card detail page. **Changing the structure costs you no marks.** What is graded is that the information listed in F-3 to F-7 is on the screen and the three states exist.

Shell (every protected page)

```
+-------------------------------------------------------------+
| FlashDeck   Dashboard   Decks                 ashik   Logout |
+-------------------------------------------------------------+
|                                                             |
|  Page title                                  [ + New deck ] |
|  ---------------------------------------------------------- |
|  page content                                               |
|                                                             |
+-------------------------------------------------------------+
```

Dashboard

```
Dashboard                                        [ + New deck ]

+----------+ +----------+ +----------+ +----------+
| Due now  | | Cards    | | Mastered | | Accuracy |
|    17    | |   128    | |    23    | |   78%    |
+----------+ +----------+ +----------+ +----------+

Where your cards sit
Box 1 |||||||||||||  40   Box 4 |||||  15
Box 2 ||||||||||||     30   Box 5 |||||||||  23
Box 3 |||||||||        20

Ready to study                                    See all decks >
+-------------------------------------------------------------+
| Django ORM          PROGRAMMING   24 cards   7 due  [ Study ]  |
| React Hooks         INTERVIEW     31 cards   4 due  [ Study ]  |
+-------------------------------------------------------------+
```

Deck detail — the cards in one deck

```
< Decks / Django ORM                          [ + Add card ] [ Study 7 ]

[ Search front or back..... ]  [ Box: All v ]

+-------------------------------------------------------------+
| Front                              Box     Due       Actions   |
|-------------------------------------------------------------|
| What does select_related() do?    [ 3 ]   in 3 days  Edit  Del |
| When is a queryset evaluated?     [ 1 ]   due now    Edit  Del |
| ...                                                          |
+-------------------------------------------------------------+
                    <  1  2  3  >            24 cards
```

Study screen

```
    Django ORM                     4 / 12    [ Exit ]
    ==========------------------------------------

    +-----------------------------------------+
    |                                         |
    |   What does select_related() do?        |
    |                                         |
    |   hint: N+1                      Box 3  |
    |                                         |
    |            [  Show answer  ]            |
    +-----------------------------------------+

    ... after the flip ...

    +-----------------------------------------+
    |   What does select_related() do?        |
    |   ------------------------------       |
    |   Follows forward FK relations with a   |
    |   SQL JOIN, in one query.               |
    |                                         |
    |      [  Missed  ]      [  Got it  ]     |
    +-----------------------------------------+
```

Delete confirmation

```
+-----------------------------------------+
|  Delete this deck?                      |
|  "Django ORM" and all 24 cards in it.   |
|  This cannot be undone.                 |
|                  [ Cancel ]  [ Delete ] |
+-----------------------------------------+
```

The three states, on every screen that loads data

```
loading  ->  skeleton rows or a spinner
empty    ->  a line of text + the button that fixes it
error    ->  "Could not load this deck."  [ Retry ]
```

The design itself is yours

Nobody should be able to tell that two submissions came from the same brief. Choose your own colours, typography, spacing, corner radius, icons, badge style and empty-state wording. Do not download a dashboard theme and re-skin it — write your own CSS (or your own Tailwind classes).

If you freeze in front of a blank page, pick one direction and commit to it:

- **Clean and calm** — white background, one accent colour, generous spacing, thin borders, no shadows.
- **Dark developer** — near-black surfaces, a single bright accent, monospace numbers, compact rows.
- **Paper and index cards** — off-white ground, a ruled-card look for the study screen, a warm accent,

soft shadows.
    Bold and editorial — big type, high contrast, colour-blocked box badges, almost no borders.

Three things are graded regardless of the direction you pick: it must be **responsive** (test at 390 px wide, not only on your laptop), the **five boxes must be distinguishable at a glance** — colour *plus* the number, so it still reads for a colour-blind user — and the study card must stay readable with a long answer in it.

PART B — OPTIONAL (0 marks — this is what makes it CV-worthy)

Nothing below affects your grade. Missing all of it costs you nothing. Doing six to eight of them is the difference between a class assignment and a portfolio project.

Backend

O-1 Tags. A `Tag` model with a many-to-many to `Card`, and `?tags=orm,queryset` on the list endpoint.

O-2 Bulk import and export. `POST /api/decks/<id>/import/` taking pasted CSV or `front,back` lines and creating the cards with `bulk_create()`, plus a CSV export of a deck. Twenty cards typed one at a time is the reason most flashcard apps die on day two.

O-3 A real review log. A `ReviewLog` model (card, correct, reviewed_at) written on every review, and `GET /api/stats/history/?days=30` returning one row per day — the honest version of `reviewed_today`.

O-4 Shareable decks. A `share_slug` on the deck, `GET /api/public/decks/<slug>/` readable with no token (`AllowAny`, cards without the review fields), and a "clone this deck into my account" endpoint.

O-5 An image on a card — `ImageField`, `MEDIA_ROOT` / `MEDIA_URL`, size and extension validation.

O-6 Throttling (anon 20/min, user 100/min) and a custom exception handler with a consistent `{success, message, data}` response shape.

O-7 Swagger / ReDoc with `drf-spectacular`, plus a Postman collection committed in the repo.

O-8 `python manage.py seed_demo` creating a demo user with 3 decks and ~60 cards spread across all five boxes, some due today — so nobody ever opens your app to an empty screen.

O-9 Tests — at least six: registration, login, owner isolation (user A cannot read user B's deck), a card cannot be posted into someone else's deck, the box goes up on correct and resets to 1 on wrong, and the stats numbers.

Frontend

O-10 Keyboard-driven study — Space flips the card, 1 is Missed, 2 is Got it, Escape exits. Add a CSS 3D flip on the card while you are there. This is what makes it feel like a real study tool.

O-11 A 30-day activity heatmap on the dashboard, GitHub-contributions style, from O-3's history endpoint. This is the screenshot that sells the project.

O-12 Charts with recharts or Chart.js — reviews per day, accuracy trend, and a doughnut of the five boxes.

O-13 Filters in the URL with `useSearchParams`, and a 400 ms debounce on the search box, so a filtered list can be shared and survives a refresh.

O-14 React Query for caching and background refetch, plus an optimistic review: the card advances the moment you click, and rolls back if the POST fails.

O-15 Dark mode toggle, and a silent token refresh in the axios response interceptor instead of kicking the user

out on a 401.

O-16 Deploy it. Backend on Render / PythonAnywhere, frontend on Vercel / Netlify. Put both live links and a demo login at the top of the READMEs, with 3–4 screenshots. A live link is worth more on a CV than any three features above it.

7. More hints

7.1 settings.py

```
INSTALLED_APPS = [
    ...,
    "rest_framework",
    "django_filters",
    "corsheaders",
    "study",              # your app
]

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",      # must be ABOVE CommonMiddleware
    "django.middleware.common.CommonMiddleware",
    ...
]

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ),
    "DEFAULT_PERMISSION_CLASSES": ("rest_framework.permissions.IsAuthenticated",),
    "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 10,
}

SIMPLE_JWT = {"ACCESS_TOKEN_LIFETIME": timedelta(days=1)}

CORS_ALLOWED_ORIGINS = ["http://localhost:5173", "http://127.0.0.1:5173"]

USE_TZ = True
TIME_ZONE = "Asia/Dhaka"
```

7.2 Register serializer and the stats view

```
class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, min_length=6)

    class Meta:
        model = User
        fields = ["id", "username", "email", "password"]

    def create(self, validated_data):
        return User.objects.create_user(**validated_data)


class StatsView(APIView):
    def get(self, request):
        cards = Card.objects.filter(deck__owner=request.user)

        rows = cards.values("box").annotate(n=Count("id"))
        boxes = {str(i): 0 for i in range(1, 6)}
        for row in rows:
            boxes[str(row["box"])] = row["n"]

        totals = cards.aggregate(r=Sum("times_reviewed"), c=Sum("times_correct"))
        reviewed, correct = totals["r"] or 0, totals["c"] or 0

        return Response({
            "decks": Deck.objects.filter(owner=request.user).count(),
            "cards": cards.count(),
```

```
            "due_now": cards.filter(next_review_at__lte=timezone.now()).count(),
            "mastered": boxes["5"],
            "reviewed_today": cards.filter(last_reviewed_at__date=timezone.localdate()).count(),
            "accuracy": round(100 * correct / reviewed) if reviewed else 0,
            "boxes": boxes,
        })
```

`if reviewed else 0` is the whole guard. Without it, every brand-new account gets a 500 on the first screen.

7.3 The two validations that protect your data

```
# DeckSerializer — unique title per user, reported under "title"
def validate_title(self, value):
    qs = Deck.objects.filter(owner=self.context["request"].user, title__iexact=value)
    if self.instance:
        qs = qs.exclude(pk=self.instance.pk)        # editing must not clash with itself
    if qs.exists():
        raise serializers.ValidationError("You already have a deck with this title.")
    return value


# CardSerializer — the deck must belong to the caller
def validate_deck(self, deck):
    if deck.owner != self.context["request"].user:
        raise serializers.ValidationError("This deck does not belong to you.")
    return deck
```

Both need `self.context["request"]`, which DRF gives you automatically inside a viewset — but not if you instantiate the serializer yourself somewhere and forget `context={"request": request}`.

7.4 The review action

```
INTERVALS = {1: 0, 2: 1, 3: 3, 4: 7, 5: 16}   # days

class CardViewSet(viewsets.ModelViewSet):
    ...
    @action(detail=True, methods=["post"])
    def review(self, request, pk=None):
        card = self.get_object()                  # 404 for someone else's card
        correct = request.data.get("correct")

        if not isinstance(correct, bool):
            return Response(
                {"correct": ["This field is required and must be true or false."]},
                status=status.HTTP_400_BAD_REQUEST,
            )

        card.box = min(card.box + 1, 5) if correct else 1
        card.next_review_at = timezone.now() + timedelta(days=INTERVALS[card.box])
        card.last_reviewed_at = timezone.now()
        card.times_reviewed += 1
        if correct:
            card.times_correct += 1
        card.save()

        return Response(self.get_serializer(card).data)
```

`isinstance(correct, bool)` is doing real work: `request.data.get("correct")` can be None, `"false"` or `0`, and two of those are truthy or falsy in ways you did not mean.

7.5 Auth context and ProtectedRoute

```
// src/auth/AuthContext.jsx
const AuthContext = createContext(null);
```

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => localStorage.getItem("username"));

  const signIn = async (credentials) => {
    const data = await login(credentials);          // src/api/auth.js
    localStorage.setItem("access", data.access);
    localStorage.setItem("refresh", data.refresh);
    localStorage.setItem("username", credentials.username);
    setUser(credentials.username);
  };

  const signOut = () => {
    localStorage.clear();
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, signIn, signOut }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);

// src/components/ProtectedRoute.jsx
export default function ProtectedRoute({ children }) {
  const token = localStorage.getItem("access");
  return token ? children : <Navigate to="/login" replace />;
}

// in the router
<Route path="/decks/:id" element={
  <ProtectedRoute><DeckDetail /></ProtectedRoute>
} />
```

7.6 Fetching a filtered, paginated list

```
const { id } = useParams();
const [data, setData] = useState({ count: 0, results: [] });
const [loading, setLoading] = useState(true);
const [error, setError] = useState(false);

useEffect(() => {
  let ignore = false;
  setLoading(true);
  setError(false);
  listCards({ deck: id, search, box, page })
    .then((res) => { if (!ignore) setData(res); })
    .catch(() => { if (!ignore) setError(true); })
    .finally(() => { if (!ignore) setLoading(false); });
  return () => { ignore = true; };
}, [id, search, box, page]);

const totalPages = Math.ceil(data.count / 10);
```

Dependencies must be primitives (`id`, `search`, `box`, `page`). Putting an object or an array literal in the dependency list gives you a new reference on every render — and an infinite request loop. When the search text or the box filter changes, call `setPage(1)` in the same handler.

7.7 The study loop

```
const [cards, setCards] = useState([]);
const [index, setIndex] = useState(0);
const [flipped, setFlipped] = useState(false);
const [saving, setSaving] = useState(false);
const [score, setScore] = useState({ done: 0, correct: 0 });
```

```
const card = cards[index];                        // undefined once the queue is finished

async function answer(correct) {
  if (saving || !card) return;                    // guards the double click
  setSaving(true);
  try {
    await reviewCard(card.id, correct);
    setScore((s) => ({ done: s.done + 1, correct: s.correct + (correct ? 1 : 0) }));
    setFlipped(false);                            // next card starts face down
    setIndex((i) => i + 1);
  } catch {
    setToast("Could not save that answer. Try again.");
  } finally {
    setSaving(false);
  }
}

if (loading) return <Loader />;
if (!cards.length) return <EmptyState />;         // nothing was due
if (!card) return <SessionSummary score={score} />;   // queue finished
```

Three renders, three states, one index. Resetting `flipped` to `false` in the same handler that advances the index is what stops the next card appearing with its answer already showing.

8. Mistakes that will cost you hours

1. **CORS. Your API works in Postman and every screen in the browser says "Network Error". Add `corsheaders` to `INSTALLED_APPS`, put `CorsMiddleware` above `CommonMiddleware`, and list your Vite origin in `CORS_ALLOWED_ORIGINS`.**
2. **Forgetting `get_queryset()`. Every user sees every user's decks, and any id can be opened by anyone. This is the most common serious bug in this kind of project, and it is checked first. For cards, remember the scope goes through the deck: `deck__owner=self.request.user`.**
3. **Trusting `deck` from the request body. Without `validate_deck()` anyone can write cards into your deck. `owner` has the same rule: set it in `perform_create()`.**
4. **Letting the client write `box` or `next_review_at`. Put them in `read_only_fields`. The ladder is the app; a client that can set its own box has no app left.**
5. **`datetime.now()` instead of `timezone.now()`. With `USE_TZ = True` you get "can't compare offset-naive and offset-aware datetimes", or worse, cards that are silently never due.**
6. **A missing trailing slash. `POST /api/cards` gets redirected to `/api/cards/` and the body is dropped. Always write the slash.**
7. **Hard-coded data in React, or `http://127.0.0.1:8000` written inside a component. The first is not wired to your API however good it looks; the second works on your laptop and nowhere else. Use `VITE_API_URL`, and restart the dev server after editing `.env`.**
8. **Committing `.env`, `db.sqlite3` or `node_modules/`. Write `.gitignore` *before* the first commit.**
9. **Showing only a toast for a 400. The user cannot tell which field is wrong. Map the DRF error object onto the fields.**
10. **No loading and no empty state — especially on the study screen, which is empty most days by design. A blank screen for two seconds reads as a broken app.**
11. **An object or an array literal in a `useEffect` dependency list — a new reference every render, so the effect never stops firing.**
12. **Dividing by zero in `accuracy`, or filtering and paginating in the browser. Fetching all the rows and**

slicing them in React defeats B-4, and falls apart the moment there are 200 cards.

9. Self-check before you submit

Every line must be true.

- ☐ Fresh clone of the backend → venv → `pip install -r requirements.txt` → `cp .env.example .env` → `migrate` → `runserver` works with no manual fixes.
- ☐ Fresh clone of the frontend → `npm install` → `.env` → `npm run dev` loads and talks to the API.
- ☐ `GET /api/decks/` without a token returns 401.
- ☐ Register two users, create a deck as user A, then request that id as user B: you get 404, not the data.
- ☐ As user B, `POST /api/cards/` with user A's deck id returns 400, not 201.
- ☐ `PATCH /api/cards/<id>/` with `{"box": 5}` does not change the box.
- ☐ `POST /api/register/` does not return the password, and the user can log in afterwards.
- ☐ `?deck=3&box=1&search=orm&ordering=-created_at&page=1` returns a sensible result set, and the response has `count`, `next`, `previous`, `results`.
- ☐ Answer a card correctly: the box goes up by one and `next_review_at` moves into the future. Answer one wrong: the box is 1 and it is due now.
- ☐ A card in Box 5 answered correctly stays at 5 — it does not become 6.
- ☐ `POST /api/cards/<id>/review/` with no body returns 400, not 500.
- ☐ `/api/stats/` on a brand-new account returns `accuracy: 0` and all five box keys — no 500, no missing keys.
- ☐ Submitting the card form empty shows a message under each bad field.
- ☐ Every screen that loads data has a loading state, an empty state and an error state (test the last one by stopping the backend).
- ☐ Typing in the search box while on page 3 takes you back to page 1.
- ☐ The study screen never shows the answer before you press Show answer, and a finished session shows the summary, not a blank page.
- ☐ Logging out and typing `/decks` sends you to `/login`; refreshing a protected page keeps you logged in.
- ☐ Delete asks for confirmation in your own modal, and the list updates without a reload.
- ☐ The whole app is usable at 390 px wide.
- ☐ `git ls-files` shows no `.env`, no `db.sqlite3`, no `node_modules/`.
- ☐ Both READMEs exist and each links to the other repo.

10. Marks (100)

Area	Marks
Models, migrations, admin, project setup	8

Area	Marks
Auth: register, login, JWT wired end to end	12
Owner scoping and permissions (401 / 404, cards scoped through the deck)	15
Deck + Card CRUD, filter, search, ordering, pagination	14
Study endpoint + review action — the box logic is correct	12
Stats endpoint	5
Frontend routing, axios instance + API layer, protected routes	10
Screens fully wired to the API (dashboard, decks, cards, study, delete)	14
Loading / empty / error states, responsive layout, your own design	6
Repo hygiene, `.env.example`, READMEs, commit history	4
Total	100

Optional items: no marks, no deductions. They can earn up to **+5 bonus** to recover marks lost elsewhere, capped at 100. Do them for your CV, not for the grade.

11. Submission

1. **Backend repo link** (public)
2. **Frontend repo link** (public)
3. **Demo login** — a username and password the grader can use (not your personal password), on an account that already has a few decks and some cards due today
4. **Live links**, if you deployed (optional)

Backend README: what the project is (and your domain, if you changed it), tech stack, setup steps, an environment variable table, the box/interval table, and an endpoint table with method + auth level.

Frontend README: setup steps, the route table (public vs protected), 3–4 screenshots including the study screen, the design direction you chose, the list of OPTIONAL items you completed, and a short "what I would add with more time".